Project Two Summary and Reflections Report

Summary

For Project One, I developed and tested three major features of a mobile application: the Contact Service, Task Service, and Appointment Service. My primary testing approach for all three features was unit testing using JUnit 5. The purpose of these tests was to verify that each class and service functioned according to the customer's requirements and properly handled both valid and invalid inputs. By creating comprehensive test cases, I was able to evaluate the behavior of each component individually before considering how they would operate within the overall application.

For the Contact Service, I created tests that verified the creation of contacts, validation of contact IDs, first names, last names, phone numbers, and addresses. I also tested the service methods responsible for adding, updating, retrieving, and deleting contacts. For the Task Service, I verified that task IDs, names, and descriptions met all specified requirements and that task records could be properly added, updated, and removed. For the Appointment Service, I tested appointment IDs, appointment descriptions, future appointment dates, and service-level functionality such as adding and deleting appointments while preventing duplicate records.

My testing approach was closely aligned with the software requirements because every requirement was translated into one or more JUnit test cases. For example, the Contact requirements specified that contact IDs could not exceed 10 characters. To verify this requirement, I created a test that attempted to create a contact with an invalid ID length and confirmed that an exception was thrown.

assertThrows(IllegalArgumentException.class,

() -> new Contact("12345678901",

"Jeremy",

"Fish",

"1234567890",

"123 Main Street"));

Likewise, the Appointment requirements specified that appointments could not be scheduled in the past. To verify this requirement, I created tests that attempted to create appointments using dates before the current date and confirmed that invalid appointments were rejected. By creating tests for both valid and invalid conditions, I ensured that the software behaved according to the stated requirements.

The overall quality of my JUnit tests was strong because they provided broad coverage across all classes and service methods. Every constructor, validation rule, update operation, add operation, delete operation, and retrieval function was tested. The tests covered both expected use cases and error conditions. While code coverage percentages alone do not guarantee software quality, the extensive coverage demonstrated that the majority of the application's functionality was exercised through automated testing. This provided confidence that defects would be identified before deployment.

Writing the JUnit tests was an important learning experience because it required me to think critically about how the software could fail instead of focusing only on how it should work. I used assertions such as assertEquals(), assertTrue(), assertFalse(), assertNull(), and

assertThrows() throughout the project. These assertions helped verify that the application returned expected results under a variety of conditions.

To ensure that my code was technically sound, I created tests that verified all required business rules. For example:

```
assertThrows(IllegalArgumentException.class,
    () -> new Appointment("1",
    pastDate(),
    "Description"));
```

This test verifies that the Appointment class properly rejects appointments scheduled in the past. By validating the requirements directly through test cases, I was able to confirm that the application enforced the constraints specified by the customer.

I also focused on keeping my test code efficient and maintainable. Most test methods were designed to verify a single requirement at a time, making failures easier to identify and troubleshoot. For example:

```
assertEquals("Jeremy", contact.getFirstName());
```

This simple assertion directly verifies one requirement without introducing unnecessary complexity. I also used service-level tests such as:

```
assertTrue(service.addContact(contact));
assertEquals(contact, service.getContact("1"));
```

These tests efficiently verify multiple aspects of service functionality while remaining easy to read and maintain.

Reflection

Testing Techniques

The primary testing technique employed throughout this project was unit testing. Unit testing focuses on evaluating individual components of software in isolation to verify that each unit performs as expected. In this project, each class and service was tested independently. This approach was especially useful because the application consisted primarily of business logic and validation rules rather than a graphical user interface.

Another testing technique I used was boundary value testing. Many of the project requirements involved maximum field lengths, such as contact IDs, task names, task descriptions, and addresses. Boundary value testing involves testing values at the edge of acceptable limits as well as values just beyond those limits. For example, when a contact ID was allowed to contain up to 10 characters, I tested IDs with exactly 10 characters and IDs with 11 characters to verify that the validation logic worked correctly.

Exception testing was another important technique used during the project. This technique verifies that invalid input data produces the expected exceptions rather than causing unexpected application behavior. Examples included testing null values, oversized strings, invalid phone numbers, and appointment dates in the past.

There were several testing techniques that were not used directly in this project. Integration testing was not necessary because the application components were tested individually rather than as an interconnected system. System testing was also not performed because the project

focused on backend services rather than a complete deployed application. User acceptance testing was not applicable because the software was not delivered to end users for evaluation. Performance testing and security testing were also outside the scope of this project because the services used in-memory storage and were not exposed to real-world traffic.

Each testing technique has practical uses depending on the type of software being developed. Unit testing is useful during development because it allows developers to identify defects early. Integration testing becomes more important when multiple systems, databases, or APIs communicate with one another. System testing helps verify that an entire application functions correctly before release. Performance testing is critical for applications expected to support large numbers of users, while security testing is essential for applications that handle sensitive information. Selecting the appropriate testing techniques helps ensure that software quality objectives are achieved efficiently.

Mindset

Throughout this project, I adopted a cautious and detail-oriented mindset while testing the application. Instead of assuming that my code was correct, I actively searched for situations where the software could fail. This approach helped me identify weaknesses in validation logic and allowed me to verify that invalid data would be rejected properly. For example, testing oversized IDs and invalid appointment dates ensured that the application enforced customer requirements consistently.

It was also important to appreciate the complexity and interrelationships of the code being tested. Although the application was relatively small, individual classes and services depended on one another. Changes to validation rules in a model class could affect service-level functionality and

corresponding test cases. Understanding these relationships helped me anticipate the impact of changes and create more comprehensive tests.

I also made a conscious effort to limit bias while reviewing my code. Developers often assume their code works because they understand the intended behavior. To reduce this bias, I intentionally created negative test cases designed to break the application. Instead of only testing successful scenarios, I tested invalid inputs, null values, duplicate IDs, and out-of-range values. These tests helped me evaluate the software objectively rather than relying on assumptions.

Bias can be a significant concern when developers test their own code because familiarity with the implementation may cause them to overlook defects. For example, a developer may focus primarily on expected use cases while ignoring uncommon error conditions. Creating comprehensive automated tests and intentionally testing invalid scenarios can help reduce this risk and improve software quality.

This project reinforced the importance of discipline and commitment to quality in software engineering. Cutting corners during development or testing may save time initially, but it often leads to technical debt that becomes more expensive to address later. For example, skipping validation tests could allow defects to remain hidden until they affect end users. These defects may require significant rework, increase maintenance costs, and damage customer confidence.

As a future software engineering professional, I plan to avoid technical debt by writing clean, maintainable code and implementing automated tests throughout the development process. I also plan to conduct regular code reviews, validate requirements carefully, and address issues as soon as they are identified. This project demonstrated that investing time in quality assurance produces more reliable software and reduces long-term maintenance challenges.

References

JUnit Team. (2024). *JUnit 5 User Guide*. https://junit.org/junit5/docs/current/user-guide/

Martin, R. C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.